Labyrinthe

BONNAZ Aymeric, MOUILLET Claude

12 septembre 2021

Nom apprenant : _____

Date de réalisation : _____

Question:	1	2	3	4	5	6	7	Total
Points:	2	2	2	2	4	5	3	20
Score:								

1 Énoncé

Depuis l'antiquité, ces amoncellements de couloirs sinueux, d'embranchements et d'impasses que l'on appelle labyrinthes exercent une profonde fascination. Ceux-ci se retrouvent sous des formes diverses dans de nombreuses civilisations à travers le monde. Le plus génial inventeur de ces motifs architecturaux est Dédale. Après avoir été chassé d'Athènes, il trouva refuge auprès du roi crétois Minos. Celui-ci lui ordonna de bâtir un endroit dans lequel le Minotaure, enfant monstrueux issu de la liaison entre sa femme Pasiphaé, rendue folle d'amour par Poséidon, et un grand taureau blanc, serait enfermé.

Le présent exercice présente la génération d'un labyrinthe représenté par une grille discrète de taille N * M cellules.
Chaque cellule contient quatre parois caractérisées par un état ouvert ou fermé (haut, bas, gauche, droite) qui la sépare des autres cellules. Une paroi fermée implique l'existence d'un mur.
Chaque cellule disposera de deux attributs supplémentaires, le premier indiquant si la cellule a déjà été visitée et l'autre s'il s'agit d'une cellule simple, de la sortie ou de l'entrée.

FIGURE 1 – Thésée vainqueur du Minotaure, Charles-Édouard Chaise, 1791

2 Exercice

Cet exercice établit un ensemble de structures et de méthodes au sujet de la modélisation des labyrinthes. La première partie aborde la génération d'un labyrinthe à partir d'une grille dont toutes les parois sont fermées, la seconde partie l'affichage de celui-ci.

Construction : 12 pts

L'objectif de cette partie est l'implémentation des différentes méthodes de la structure de données **Maze** définie ci-dessous.
Dans celle-ci, la grille de taille N * M est représentée par un tableau de cellules **Cell** à deux dimensions. La cellule **(0,0)** représente le coin supérieur gauche et la cellule **(N-1, M-1)** le coin inférieur droit. Chaque cellule dispose d'un tableau de quatre booléens déterminant l'état de ces parois (haut, bas, gauche, droit), d'un booléen décrivant si elle a été visitée et une valeur indiquant son statut.

Maze :

- **IsOpen**(int i, int j, int w) : bool
- **IsMazeStart**(int i, int j) : bool
- **IsMazeEnd**(int i, int j) : bool
- **Open**(int i, int j, int w) : void
- **Generate**() : KeyValuePair<int, int>
- **CloseNeighbors**(int i, int j) : List<KeyValuePair<int, int> >
- **DisplayLine**(int n) : string
- **Display**() : List<string>

1. Structure d'une cellule
   (a) (1 point) Définir une structure de données **Cell** à l'aide des informations fournies précédemment.
   (b) (1 point) Quelle représentation a été choisie pour représenter le statut ?
   Justifier votre réponse :

   _____

   _____

2. Propriétés du système :
   (a) (1 point) Est ce que la paroi w de la cellule (i, j) est ouverte/fermée ?
   Implémenter la méthode **IsOpen**(int i, int j, int w) : bool.
   (b) (1 point) Est-ce que la cellule (i, j) est l'entrée ou la sortie du labyrinthe ?
   Implémenter les méthodes **IsMazeStart**(int i, int j) : bool et **IsMazeEnd**(int i, int j) : bool.

L'action unitaire à réaliser lors de la génération d'un labyrinthe est l'ouverture d'une paroi située entre deux cellules.

3. (2 points) Implémenter la méthode d'ouverture de la paroi w de la cellule (i, j) :
   Open(int i, int j, int w) : void.

Suggestion : Faire attention à la paroi miroir de l'autre cellule.

Afin de correctement de compléter la méthode **Generate**(), il est important de déterminer l'ensemble des voisins d'une cellule.

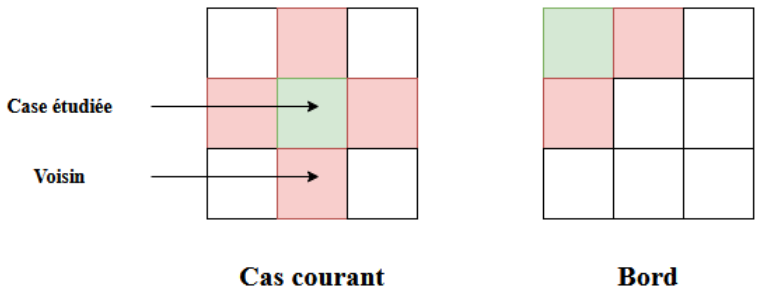

FIGURE 2 – Détermination des voisins d'une cellule de la grille

4. (2 points) Implémenter la méthode déterminant les proches voisins :
   CloseNeighbors(int i, int j) : List<KeyValuePair<int, int> >.

Suggestion : Faire attention aux effets de bords.

La génération du labyrinthe est exécutée de la manière suivante :

- L'ensemble des cellules sont à l'état non visité et les parois sont toutes fermées.
- Choisir aléatoirement une cellule de la grille et mettre son état de visite à **true**.
- Choisir aléatoirement parmi ses cellules voisines proches non visitées.
- Ouvrir la paroi entre les deux cellules, mettre l'état de visite de la cellule choisie à **true** et s'y placer.
- Répéter les deux dernières étapes jusqu'à ce que la cellule étudiée n'ait plus de voisins non visités.
- Revenir sur la cellule précédente et répéter les trois dernières opérations.
- Lorsque la cellule précédente est la première cellule, choisir deux cellules du bord aléatoirement pour signifier l'entrée et la sortie du labyrinthe.

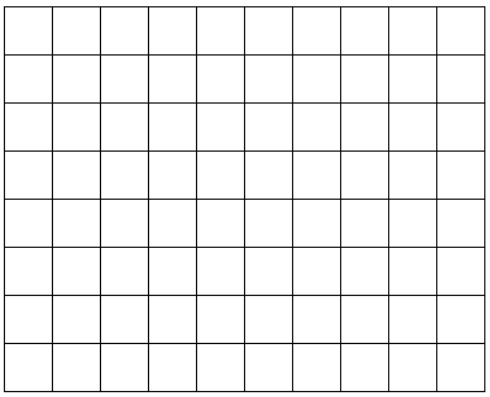

FIGURE 3 – Grille initiale 8 * 10

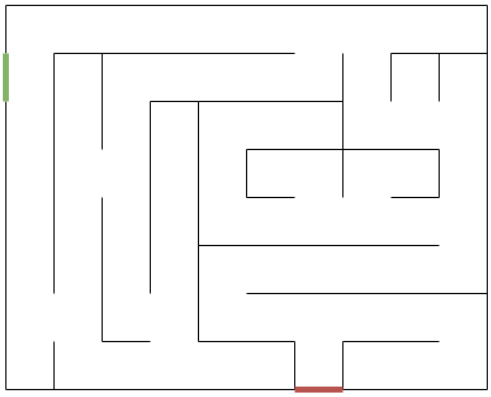

FIGURE 4 – Labyrinthe généré sur cette grille

5. Génération du labyrinthe dont la paire d'entiers de retour désigne l'entrée :
   (a) (3 points) Implémenter la méthode **Generate**() : KeyValuePair<int, int>.
   (b) (1 point) Expliquer, intuitivement, pour quelles raisons l'algorithme précédent visite l'ensemble des cellules du labyrinthe ?

   _____

   _____

   _____

Suggestion : Afin d'éviter un nombre important d'appels récursifs, l'utilisation d'une pile (**stack**) est possible.

Affichage : 8 pts

L'objectif de cette partie est l'affichage dans un fichier du schéma du labyrinthe généré. Cette section implémente les deux méthodes retournant les chaînes de caractères représentant graphiquement le labyrinthe généré dans la première partie.

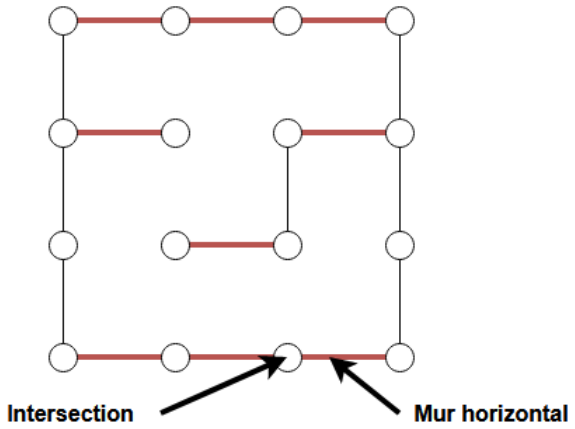

FIGURE 5 – Labyrinthe 3 * 3 avec mise en évidence des murs à afficher et des intersections.

Deux éléments sont représentés par l'affichage du labyrinthe. Il s'agit des intersections entre cellules et les murs horizontaux représentés respectivement comme de grands ronds et des lignes rouges dans la figure ci-dessus. Il y a N + 1 lignes de caractères à représenter. La liste des caractères utilisables est fournie ci-dessous.

```
Liste des intersections =[' ','-','╵', '╷','╶','═','╶','╴','╹','╻','│','┤','└','┴','├','┼']
```

FIGURE 6 – Liste des caractères fournis pour l'affichage du labyrinthe.

Chaque intersection a deux, trois ou quatre parois de cellules. Une paroi de cellule est un mur (**false**) ou une absence de mur (**true**) (la grille de cellules permet d'obtenir ces informations). Chaque combinaison d'états des parois est toujours représentée par un unique caractère.
Chaque ligne horizontale est soit un espace vide, soit une barre horizontale.

6. (5 points) Détermination utilisation des caractères :
   Établir pour chaque caractère (ou caractères similaires) les combinaisons sur lesquelles il est utilisé (un fichier d'exemple de labyrinthe est fourni).

   _____
   _____
   _____
   _____
   _____
   _____
   _____
   _____

7. Implémentations :
   (a) (2 points) Quelle est la chaîne de caractère représentant la ligne n ?
   Implémenter la méthode **DisplayLine**(int n) : string.
   (b) (1 point) Quelle est la liste des chaînes de caractères donnant le labyrinthe ?
   Implémenter la méthode **Display**() : List<string>.